

Documentação do Bot de Trading (Spot Trading)

1. Estrutura Geral:

O bot é construído com uma arquitetura modular, dividindo a lógica em vários arquivos (módulos) que interagem entre si, com um arquivo **main.py** sendo a peça central (o "cérebro" que orquestra todas as ações). Os módulos do código são responsáveis por:

Comunicação com a API: Lidar com as interações com a Binance (buscar dados, criar ordens, etc.).

Cálculos: Calcular indicadores técnicos (RSI, MACD, etc.) e analisar padrões de velas.

Lógica de Decisão: Decidir se uma ordem de compra ou venda deve ser feita, com base nas análises.

Gestão de Ordens OCO: Lidar com a criação e acompanhamento das ordens (ordem OCO).

Notificações: Gerenciar mensagens no **Telegram**.

Registro e Armazenamento: Gerar e salvar um relatório em um arquivo Excel.

2. Arquivos e Funções (Módulos):

config.py:

Função: Este arquivo armazena todas as configurações do bot, incluindo chaves de API (para Binance e **Telegram**), parâmetros dos indicadores, níveis de **stop loss** e **take profit**, e outros parâmetros operacionais e de análise (períodos para cálculo, limites para consulta de dados etc).

Interação: É importado e usado por todos os outros módulos, sendo que ele é sempre lido antes que qualquer outra função seja executada (pois a primeira coisa a acontecer, no seu **main.py**, é chamar as funções de sincronização de horário e inicialização do bot).

binance_api.py:

Função: Lida com todas as interações com a API da Binance, incluindo buscar saldos, preços de fechamento das velas, volume, ordem books e detalhes de ordens OCO.

Funções:

get_usdt_balance(client): Obtém o saldo disponível em USDT na conta da Binance.

get_order_book(client, symbol, depth): Recupera o livro de ofertas (ordens de compra e venda) de um ativo específico.

get_order_details(client, symbol, order_id): Obtém informações sobre uma ordem específica (como status, preço etc).

get_closes(client, symbol, interval, limit): Pega a lista dos preços de fechamento das velas do ativo em um período de tempo definido.

get_volumes(client, symbol, interval, limit): Obtém os volumes de negociação das velas do ativo.

Interação: Todas as funções desse modulo são usadas em **main.py**, por outras funções do **trading_functions.py** ou mesmo em **decision.py**.

trading_functions.py:

Função: Contém a lógica para calcular diversos indicadores técnicos, analisar padrões de velas e checar a tendência, além de gerenciar algumas operações como os cálculos de venda.

Funções:

calculate_sell_pressure(order_book): Calcula a pressão de venda com base no livro de ofertas.

calculate_moving_average_sell_pressure(client, symbol, interval, limit, depth): Calcula a média móvel da pressão de venda.

calculate_rsi(closes, period): Calcula o Índice de Força Relativa (RSI).

calculate_macd(closes, slow, fast, signal): Calcula a média móvel de convergência e divergência (MACD) e sua linha de sinal.

calculate_bollinger_bands(closes, period, num_std): Calcula as Bandas de Bollinger (inferior, média e superior).

calculate_volume_moving_average(volumes, period): Calcula a média móvel do volume.

calculate_trade_result(purchase_price, executed_qty, sell_price): Calcula o lucro ou prejuízo de uma operação.

calculate_moving_average(closes, period): Calcula a média móvel dos preços de fechamento.

check_trend(client, symbol, interval, short_period, long_period): Verifica se a tendência é de alta.

check_candle_patterns(client, symbol, interval, limit): Detecta padrões de candle e sinaliza se eles são favoráveis para compra, venda ou nenhuma das ações.

calculate_vwap(closes, volumes): Calcula o vwap

get_candle_details(client, symbol, interval, limit): Retorna os valores do candle atual (abertura, máxima, mínima, fechamento e volume).

calculate_ema(closes, period): Calcula a Média Móvel Exponencial do período especificado.

Funções para verificar os padrões de candle como **is_hammer**, **is_shooting_star**, **is_bullish_engulfing**, **is_piercing_line**, **is_dark_cloud_cover** etc. (que definem a lógica de cada padrão)

Interação: Usado pelo **main.py**, **decision.py**, e **post_trade.py** para análises e cálculos.

patterns.py:

Função: Contém a lógica para a leitura de diversos padrões de candles, sendo eles chamados pela função **check_candle_patterns**.

Funções: **is_hammer**, **is_shooting_star**, **is_bullish_engulfing**, **is_piercing_line**, **is_dark_cloud_cover**, **is_kicker_bullish**, **is_kicker_bearish**, **is_long_day**, **is_short_day**, **is_doji**, **is_doji_dragonfly**, **is_doji_gravestone**, **is_doji_long_shadows**, **is_bullish_and_bearish_strike**, **is_rising_three_methods**, **is_falling_three_methods** e **is_stick_sandwich**.

Interação: Usado pela função **check_candle_patterns** em **trading_functions.py** para realizar o filtro de velas que dão suporte ao seu trading.

decision.py:

Função: Define a lógica de tomada de decisão do bot, baseado nos resultados dos indicadores calculados pelo **trading_functions.py**.

Funções:

should_place_order(client, symbol, SELL_PRESSURE_THRESHOLD, interval, limit): Decide se uma ordem pode ser colocada, verificando a pressão de venda.

should_buy(rsi, trend_is_up, candle_decision, macd_current, signal_line_current, last_close, lower_band, vwap): Verifica as condições para compra, e retorna um sinal lógico e string informando a lógica usada

should_sell(rsi, trend_is_up, candle_decision, macd_current, signal_line_current, last_close, lower_band, vwap): Verifica as condições para venda.

adjust_price_to_tick_size(price, tick_size): Ajusta um preço para um múltiplo de tamanho de tick, o valor mínimo de um ativo que ele consegue variar.

adjust_quantity_to_lot_size(quantity, symbol_info): Ajusta a quantidade da moeda para que respeite as regras da binance.

calculate_adjustment(current_price, current_quantity, required_notional, current_notional): Calcula ajustes que respeitem a mínima operação aceita pela binance

get_min_notional(symbol_info): Define o valor notional para o par de moedas que você está operando (o valor mínimo para ter uma operação aceita)

adjust_and_place_oco_order(client, symbol, quantity, tick_size, min_price_move): Cria e configura as ordens OCO no momento da compra (usando todos os cálculos acima para os valores dos parâmetros das ordens OCO)

adjust_rsi_levels(result): Adapta os níveis do RSI conforme o resultado dos seus trades.

Interação: Usado pelo [main.py](#) para definir as condições de entrada no mercado, para gerar as ordens OCO e para a adaptação dos níveis de RSI.

[post_trade.py:](#)

Função: Processa o resultado das ordens, envia mensagens de notificação para o [Telegram](#), armazena os resultados em um relatório Excel.

Funções:

process_order_details(limit_order_details, stop_order_details, price, executed_qty, quantia_usdt_investimento_inicial): Analisa os resultados das ordens e atualiza o saldo de acordo.

log_and_notify_results(order_result, symbol, trade_result, total_difference, timestamp, vwap, adx): Imprime os resultados no terminal, envia as notificações no [telegram](#) e chama a função para salvar no excel.

create_data_row(order_count, saldo_inicial_usdt, quantia_usdt_investimento_inicial, symbol, executed_qty, price_rounded, purchase_timestamp, lucro_alvo, stop_loss, stop_limit, order_result, oco_timestamp, trade_result, total_difference, saldo_atual_usdt, rsi_value, executed_condition, vwap, adx): Cria um dicionário com dados detalhados de cada operação (valores, parâmetros, indicadores, tudo) para facilitar na criação de uma linha da planilha do relatório.

save_to_excel(data_row): Salva os resultados em uma planilha Excel (na pasta do projeto), com todas as colunas definidas na função acima

Interação: Chamado pelo [main.py](#) depois que as ordens são executadas, gerando os logs e a planilha excel.

[pre_start.py:](#)

Função: Contém funções para sincronizar o horário, escolher o ativo que será utilizado e cancelar ordens antigas do robô

Funções:

read_last_sync_time(): Lê o ultimo horário da sincronização do sistema no arquivo last_sync_time.txt .

write_last_sync_time(): Escreve o horário atual no arquivo last_sync_time.txt após realizar a sincronização com a internet.

synchronize_time(): Faz a sincronização do horário do sistema com a internet, para evitar erros no timestamp com a api da binance.

cancel_all_oco_orders(client, symbol): Cancela todas as ordens OCO em aberto no símbolo escolhido.

escolher_simbolo(): Pergunta pro usuário qual ativo ele quer usar, exibindo uma lista ou deixando o usuário digitar um par de moedas específico.

Interação: Esse modulo é executado no início do seu [main.py](#), para preparar tudo para as operações.

telegram_integration.py:

Função: Responsável por enviar mensagens ao Telegram.

Funções:

send_telegram_message(bot_token, chat_id, message): Envia mensagens para o canal especificado, utilizando dados do seu bot e tokens do seu bot do telegram.

Interação: Usado para mandar mensagens nos arquivos **main.py**, **decision.py** e **post_trade.py**.

formatting.py:

Função: Define cores, emojis e padrões de texto para ser utilizado no console e nos seus envios para telegram, isso pode ser importante no dia a dia para que as mensagens fiquem claras, diretas e objetivas.

Variáveis: Emojis (como ALERT, STOP, etc.) e formatações de texto (como RED, GREEN, etc) para exibir resultados no console. Tem formatações em HTML também, para serem utilizadas nas mensagens do telegram.

Interação: Utilizado em todos os scripts que exibem mensagens na tela (ou que enviam mensagens via telegram), principalmente **main.py**, **decision.py** e **post_trade.py**

requirements.txt:

Função: Contém todos os pacotes utilizados pelo seu script, para o python realizar o download automático (através do comando pip) ao usar pela primeira vez (ou para garantir que seus pacotes estejam sempre atualizados com `--upgrade pip`)

Interação: Utilizado pelo python no seu ambiente virtual ao usar o comando `pip install -r requirements.txt`.

3. main.py:

Função: É o arquivo principal que coordena todas as operações do bot, inicializa o cliente da Binance, configura o gerenciador de sockets, implementa os loops de negociação, gerencia exceções e executa os outros módulos. O código executa as funções de inicialização, escuta ordens no socket da binance e faz todas as checagens antes de uma compra (ou venda).

A lógica da tomada de decisão: É feita através do uso da função **should_place_order**, para verificar a pressão de vendas no momento atual do ativo, e a `should_buy` para então executar uma compra usando suas configurações.

As ordens OCO: As funções desse modulo são também responsáveis pela criação e execução de ordens de compra ou venda com stoploss e gain (OCO)

Execução de funções e loop principal Dentro de um loop, o bot verifica continuamente o saldo, executa indicadores e executa ordens. Além disso, faz tratamento de erros, e implementa pausas na operação quando muitos stops acontecem em sequência

Mensagens via telegram: envia notificações, registra logs e cria os relatórios em excel

Fluxo de Funcionamento do Bot:

Início: O **main.py** sincroniza o tempo do sistema e cria um cliente com a Binance.

Seleção do Ativo: O usuário escolhe o ativo que deseja operar.

Loop Principal:

O bot verifica continuamente o saldo da conta em USDT e pega os dados da API da Binance (velas, volume, preço).

Calcula e verifica os valores de RSI, MACD, Bandas de Bollinger, VWAP, Tendências e padrões de candles para então definir se fará uma ação ou se continuará verificando os dados

A função **should_place_order** é responsável por verificar se o valor da pressão de venda está adequado para compra.

Se a pressão estiver baixa, a função **should_buy** analisa seus parâmetros e (se eles estiverem corretos), o robô realiza a compra usando o saldo atual (com lógica de juros compostos) e armazena as informações no terminal e salva para a criação do relatório final, depois usa o código do **adjust_and_place_oco_order**, para criar a ordem OCO (lucro + stop loss).

A informação dessa compra é logada no terminal, mandada no **telegram**, e colocada em sua planilha excel.

O socket da binance verifica se a ordem OCO foi feita e pega as informações dela e, baseado no resultado (lucro ou stop), a função **process_order_details** faz os cálculos necessários (e ajusta os níveis de RSI com o **adjust_rsi_levels**) e depois os logs e a planilha são salvos e atualizados.

* O loop continua em busca de novas ordens.

Gerenciamento de Erros: Caso ocorra algum erro na comunicação com a API da Binance, o bot tenta se conectar novamente por algumas vezes (ou encerra, caso não seja possível)

Fim: Quando o programa é finalizado (ou ocorre algum problema muito grave), a conexão com a API é fechada corretamente e o bot é finalizado.

Integrações Chave:

O **main.py** usa as funcionalidades definidas na sua função **should_buy** do arquivo **decision.py**, com base nos resultados do seu módulo **trading_functions.py** (e funções de leitura das velas do **patterns.py**)

As funções de envio para **telegram** de **telegram_integration.py** são usadas em diferentes momentos do script, para você acompanhar tudo em tempo real.

O arquivo **post_trade.py** (que é chamado depois da operação ser finalizada) utiliza também os dados recebidos pela API, através do **binance_api.py**, para gerar as linhas do relatório que será salvo na sua planilha em formato xlsx.